

MT5 Trading Bots - Complete Source

Repository: github.com/Miten001/earn

Files included:

- requirements.txt - Python dependencies
- smc_ict_bot.py - SMC/ICT strategy bot (recommended)
- mt5_final_bot.py - Multi-Timeframe trend following bot
- mt5_bot.py - Scalping + Triangular arbitrage bot
- config.py - Shared config for v1 bot

Setup steps:

1. Install MT5 terminal on Windows and login (broker demo recommended).
2. pip install MetaTrader5 pandas numpy
3. Edit MT5_LOGIN / MT5_PASSWORD / MT5_SERVER at top of any bot file.
4. Run: python <botfile>.py

Path in repo: /earn_bot.py

[illegible]

```

SWEEP_LOOKBACK_M15 = 30
ATR_PERIOD          = 14
SL_BUFFER_ATR       = 0.3

# Trailing
BREAKEVEN_AT_R = 1.0
LOCK_HALF_AT_R = 1.5
TRAIL_START_R  = 2.0
ATR_TRAIL_MULT = 1.5

# Order params
PENDING_ORDER_EXPIRY_HOURS = 4
MAX_SPREAD_FX              = 25
MAX_SPREAD_CRYPT0         = 5000

# Loop
SCAN_INTERVAL_SEC = 10
STATE_FILE         = "earn_bot_state.json"
TRADES_LOG_FILE    = "earn_bot_trades.json"

# =====

# ---- Imports ----
try:
    import numpy as np
    import pandas as pd
except ImportError:
    print("[!] pip install numpy pandas")
    sys.exit(1)

try:
    import MetaTrader5 as mt5
except ImportError:
    print("[!] pip install MetaTrader5")
    sys.exit(1)

try:
    import tkinter as tk
    from tkinter import ttk
    HAS_TK = True
except ImportError:
    HAS_TK = False

try:
    import matplotlib
    matplotlib.use("TkAgg")
    import matplotlib.pyplot as plt
    from matplotlib.figure import Figure
    from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
    from matplotlib.patches import Rectangle
    HAS_MPL = True
except ImportError:
    HAS_MPL = False

# =====
# Indicators
# =====
def atr(df, p=14):
    h, l, c = df["high"], df["low"], df["close"]
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    return tr.ewm(alpha=1/p, adjust=False).mean()

# =====
# MT5 helpers
# =====
TF_MAP = {
    "M5": mt5.TIMEFRAME_M5, "M15": mt5.TIMEFRAME_M15,
    "H1": mt5.TIMEFRAME_H1, "H4": mt5.TIMEFRAME_H4,
}

def mt5_connect():
    kw = {"login": MT5_LOGIN, "password": MT5_PASSWORD, "server": MT5_SERVER}

```

```

    if MT5_PATH:
        kw["path"] = MT5_PATH
    if not mt5.initialize(**kw):
        return None, f"init failed: {mt5.last_error()}"
    acc = mt5.account_info()
    if acc is None:
        return None, f"login failed: {mt5.last_error()}"
    return acc, "ok"

def find_symbol(base):
    for cand in [base, base + "m", base + ".r", base + "_", base + "!", base + ".s"]:
        if mt5.symbol_info(cand) is not None:
            mt5.symbol_select(cand, True)
            return cand
    matches = mt5.symbols_get(f"*{base}*")
    if matches:
        mt5.symbol_select(matches[0].name, True)
        return matches[0].name
    return None

def get_rates(sym, tf, n=300):
    bars = mt5.copy_rates_from_pos(sym, TF_MAP[tf], 0, n)
    if bars is None or len(bars) < 50:
        return None
    df = pd.DataFrame(bars)
    df["time"] = pd.to_datetime(df["time"], unit="s")
    return df

def is_crypto(s):
    return any(c in s.upper() for c in ["BTC", "ETH", "XRP", "LTC", "DOGE", "SOL"])

def in_killzone():
    h = datetime.now(timezone.utc).hour
    return (LONDON_KZ[0] <= h < LONDON_KZ[1]) or (NEWYORK_KZ[0] <= h < NEWYORK_KZ[1])

def spread_points(sym):
    info = mt5.symbol_info(sym)
    return info.spread if info else 9999

def open_count(sym=None):
    pos = mt5.positions_get(symbol=sym) if sym else mt5.positions_get()
    if pos is None:
        return 0
    return sum(1 for p in pos if p.magic == MAGIC_NUMBER)

def pending_count(sym=None):
    orders = mt5.orders_get(symbol=sym) if sym else mt5.orders_get()
    if orders is None:
        return 0
    return sum(1 for o in orders if o.magic == MAGIC_NUMBER)

# =====
# SMC analysis
# =====
def mark_pivots(df, lb):
    df = df.copy()
    df["ph"] = False
    df["pl"] = False
    for i in range(lb, len(df) - lb):
        wh = df["high"].iloc[i - lb:i + lb + 1]
        wl = df["low"].iloc[i - lb:i + lb + 1]
        if df["high"].iloc[i] == wh.max() and (wh == df["high"].iloc[i]).sum() == 1:
            df.iat[i, df.columns.get_loc("ph")] = True
        if df["low"].iloc[i] == wl.min() and (wl == df["low"].iloc[i]).sum() == 1:
            df.iat[i, df.columns.get_loc("pl")] = True
    return df

```

```

def htf_bias(sym):
    df = get_rates(sym, "H1", 250)
    if df is None:
        return None
    df = mark_pivots(df, PIVOT_LOOKBACK_H1)
    phs = df[df["ph"]].tail(3)["high"].values
    pls = df[df["pl"]].tail(3)["low"].values
    if len(phs) < 2 or len(pls) < 2:
        return None
    if phs[-1] > phs[-2] and pls[-1] > pls[-2]:
        return "UP"
    if phs[-1] < phs[-2] and pls[-1] < pls[-2]:
        return "DOWN"
    return None

def detect_sweep(df_m15, direction):
    last = df_m15.iloc[-2]
    window = df_m15.iloc[-(SWEEP_LOOKBACK_M15 + 2):-2]
    if len(window) < 10:
        return None
    if direction == "UP":
        prior_low = window["low"].min()
        if last["low"] < prior_low and last["close"] > prior_low:
            return {"swept_level": prior_low, "sweep_low": last["low"]}
    else:
        prior_high = window["high"].max()
        if last["high"] > prior_high and last["close"] < prior_high:
            return {"swept_level": prior_high, "sweep_high": last["high"]}
    return None

def find_fvg(df, direction, min_idx=0):
    fvg = []
    for i in range(max(2, min_idx), len(df)):
        c1, c3 = df.iloc[i - 2], df.iloc[i]
        if direction == "UP" and c1["high"] < c3["low"]:
            fvg.append({"top": c3["low"], "bot": c1["high"],
                        "mid": (c3["low"] + c1["high"]) / 2, "idx": i})
        if direction == "DOWN" and c1["low"] > c3["high"]:
            fvg.append({"top": c1["low"], "bot": c3["high"],
                        "mid": (c1["low"] + c3["high"]) / 2, "idx": i})
    if not fvg:
        return None
    last_close = df.iloc[-1]["close"]
    for f in reversed(fvg):
        if direction == "UP" and last_close > f["top"]:
            return f
        if direction == "DOWN" and last_close < f["bot"]:
            return f
    return None

def detect_choch_m5(df, direction, sweep_idx):
    pdf = mark_pivots(df, PIVOT_LOOKBACK_M15)
    after = pdf.iloc[sweep_idx:]
    if direction == "UP":
        recent = after[after["ph"]].tail(1)
        if recent.empty:
            return False
        return df.iloc[-1]["close"] > recent["high"].iloc[0]
    else:
        recent = after[after["pl"]].tail(1)
        if recent.empty:
            return False
        return df.iloc[-1]["close"] < recent["low"].iloc[0]

def find_target_liquidity(df_h1, direction, entry):
    pdf = mark_pivots(df_h1, PIVOT_LOOKBACK_H1)
    if direction == "UP":
        future = pdf[pdf["ph"] & (pdf["high"] > entry)]["high"]
        if not future.empty:
            return float(future.min())

```

```

        return float(pdf["high"].tail(50).max())
    else:
        future = pdf[pdf["pl"] & (pdf["low"] < entry)]["low"]
        if not future.empty:
            return float(future.max())
        return float(pdf["low"].tail(50).min())

# =====
# Position sizing & orders
# =====
def calc_lot(sym, entry, sl, risk_pct):
    acc = mt5.account_info()
    info = mt5.symbol_info(sym)
    if acc is None or info is None:
        return 0.0
    risk_money = acc.balance * (risk_pct / 100.0)
    sl_dist = abs(entry - sl)
    if sl_dist <= 0:
        return 0.0
    ts = info.trade_tick_size or info.point
    tv = info.trade_tick_value
    if not ts or not tv:
        return 0.0
    money_per_lot = (sl_dist / ts) * tv
    if money_per_lot <= 0:
        return 0.0
    lot = risk_money / money_per_lot
    step = info.volume_step or 0.01
    lot = round(lot / step) * step
    lot = max(info.volume_min, min(info.volume_max, lot))
    return round(lot, 2)

def place_pending_limit(sym, side, lot, price, sl, tp, comment):
    info = mt5.symbol_info(sym)
    if info is None or lot <= 0:
        return None
    digits = info.digits
    otype = mt5.ORDER_TYPE_BUY_LIMIT if side == "BUY" else mt5.ORDER_TYPE_SELL_LIMIT
    expiry = int((datetime.now() + timedelta(hours=PENDING_ORDER_EXPIRY_HOURS)).timestamp())
    req = {
        "action": mt5.TRADE_ACTION_PENDING,
        "symbol": sym, "volume": lot, "type": otype,
        "price": round(price, digits), "sl": round(sl, digits), "tp": round(tp, digits),
        "deviation": 30, "magic": MAGIC_NUMBER, "comment": comment[:31],
        "type_time": mt5.ORDER_TIME_SPECIFIED, "expiration": expiry,
        "type_filling": mt5.ORDER_FILLING_RETURN,
    }
    r = mt5.order_send(req)
    if r is None or r.retcode != mt5.TRADE_RETCODE_DONE:
        return None
    return r.order

def modify_sl(pos, new_sl):
    info = mt5.symbol_info(pos.symbol)
    digits = info.digits if info else 5
    req = {
        "action": mt5.TRADE_ACTION_SLTP,
        "position": pos.ticket, "symbol": pos.symbol,
        "sl": round(new_sl, digits), "tp": round(pos.tp, digits),
    }
    r = mt5.order_send(req)
    return r is not None and r.retcode == mt5.TRADE_RETCODE_DONE

# =====
# State persistence
# =====
def load_state():
    if os.path.exists(STATE_FILE):
        try:
            with open(STATE_FILE) as f:
                return json.load(f)

```

```

        except Exception:
            pass
    return {}

def save_state(s):
    try:
        with open(STATE_FILE, "w") as f:
            json.dump(s, f)
    except Exception:
        pass

def append_trade_log(entry):
    log = []
    if os.path.exists(TRADES_LOG_FILE):
        try:
            with open(TRADES_LOG_FILE) as f:
                log = json.load(f)
        except Exception:
            log = []
    log.append(entry)
    with open(TRADES_LOG_FILE, "w") as f:
        json.dump(log, f)

def load_trade_log():
    if os.path.exists(TRADES_LOG_FILE):
        try:
            with open(TRADES_LOG_FILE) as f:
                return json.load(f)
        except Exception:
            pass
    return []

# =====
# Bot Engine (background)
# =====
class BotEngine:
    def __init__(self, log_callback=None):
        self.symbols = []
        self.state = load_state()
        self.log_callback = log_callback or (lambda m: print(m))
        self.running = False
        self.thread = None
        self.last_scan_info = {}    # {symbol: {bias, sweep_active, fvg_active, ...}}

    def log(self, msg):
        ts = datetime.now().strftime("%H:%M:%S")
        full = f"[{ts}] {msg}"
        self.log_callback(full)

    def resolve_symbols(self):
        self.symbols = []
        for s in SYMBOLS_FOREX + SYMBOLS_CRYPTO:
            actual = find_symbol(s)
            if actual:
                self.symbols.append(actual)
                self.log(f"  {s} -> {actual}")
            else:
                self.log(f"  {s} not available")

    def try_setup(self, sym):
        info_state = {"symbol": sym, "bias": None, "sweep": False,
                     "choch": False, "fvg": False, "rrr": None}
        try:
            if open_count() + pending_count() >= MAX_OPEN_TRADES:
                return info_state
            if open_count(sym) + pending_count(sym) >= MAX_TRADES_PER_PAIR:
                return info_state
            if not is_crypto(sym) and not in_killzone():
                return info_state
            cap = MAX_SPREAD_CRYPTO if is_crypto(sym) else MAX_SPREAD_FX
            if spread_points(sym) > cap:

```

```

        return info_state

    direction = htf_bias(sym)
    info_state["bias"] = direction
    if direction is None:
        return info_state

    df_m15 = get_rates(sym, "M15", 200)
    if df_m15 is None:
        return info_state
    sweep = detect_sweep(df_m15, direction)
    info_state["sweep"] = sweep is not None
    if sweep is None:
        return info_state

    df_m5 = get_rates(sym, "M5", 200)
    if df_m5 is None:
        return info_state
    sweep_idx_m5 = max(0, len(df_m5) - 30)
    choch = detect_choch_m5(df_m5, direction, sweep_idx_m5)
    info_state["choch"] = choch
    if not choch:
        return info_state

    fvg = find_fvg(df_m5, direction, sweep_idx_m5)
    info_state["fvg"] = fvg is not None
    if fvg is None:
        return info_state

    atr_m15 = atr(df_m15, ATR_PERIOD).iloc[-1]
    if pd.isna(atr_m15):
        return info_state
    entry = fvg["mid"]
    if direction == "UP":
        sl = sweep["sweep_low"] - SL_BUFFER_ATR * atr_m15
    else:
        sl = sweep["sweep_high"] + SL_BUFFER_ATR * atr_m15
    risk_dist = abs(entry - sl)
    if risk_dist <= 0:
        return info_state

    df_h1 = get_rates(sym, "H1", 200)
    target = find_target_liquidity(df_h1, direction, entry) if df_h1 is not None else None
    if target is None:
        target = entry + 2 * risk_dist if direction == "UP" else entry - 2 * risk_dist

    rr = abs(target - entry) / risk_dist
    info_state["rr"] = round(rr, 2)
    if rr < MIN_RR_TO_TARGET:
        return info_state

    side = "BUY" if direction == "UP" else "SELL"
    lot = calc_lot(sym, entry, sl, RISK_PERCENT)
    if lot <= 0:
        return info_state

    ticket = place_pending_limit(sym, side, lot, entry, sl, target,
                                f"SMC_{direction}_RR{rr:.1f}")
    if ticket is None:
        return info_state

    self.state[str(ticket)] = {
        "type": "pending", "symbol": sym, "side": side,
        "entry": entry, "initial_sl": sl, "tp": target,
        "r_distance": risk_dist, "stage": 0,
        "opened_at": datetime.now().isoformat(),
    }
    save_state(self.state)
    self.log(f"[+] {sym} {side} LIMIT lot={lot} @ {entry:.5f} SL={sl:.5f} TP={target:.5f} RR={rr
:.2f}")
    except Exception as e:
        self.log(f"[!] {sym} error: {e}")
    return info_state

def manage_open(self):

```



```

positions = mt5.positions_get() or []
pos_tickets = {str(p.ticket): p for p in positions if p.magic == MAGIC_NUMBER}

# Promote pending -> position
for tk, st in list(self.state.items()):
    if st.get("type") == "pending" and tk in pos_tickets:
        st["type"] = "position"
        self.log(f"[*] Pending #{tk} ({st['symbol']}) FILLED")
        save_state(self.state)

# Trail logic
for pos in positions:
    if pos.magic != MAGIC_NUMBER:
        continue
    st = self.state.get(str(pos.ticket))
    if st is None:
        continue
    rd = st.get("r_distance", 0)
    if rd <= 0:
        continue
    tick = mt5.symbol_info_tick(pos.symbol)
    if tick is None:
        continue
    cur = tick.bid if pos.type == mt5.ORDER_TYPE_BUY else tick.ask
    entry = st["entry"]
    profit_r = (cur - entry) / rd if pos.type == mt5.ORDER_TYPE_BUY else (entry - cur) / rd
    if profit_r <= 0:
        continue
    new_sl = pos.sl
    stage = st.get("stage", 0)
    if profit_r >= BREAK EVEN_AT_R and stage < 1:
        new_sl = entry
        st["stage"] = 1
    if profit_r >= LOCK_HALF_AT_R and stage < 2:
        new_sl = entry + 0.5 * rd if pos.type == mt5.ORDER_TYPE_BUY else entry - 0.5 * rd
        st["stage"] = 2
    if profit_r >= TRAIL_START_R:
        df = get_rates(pos.symbol, "M15", 50)
        if df is not None:
            a = atr(df, ATR_PERIOD).iloc[-1]
            if pos.type == mt5.ORDER_TYPE_BUY:
                chand = cur - ATR_TRAIL_MULT * a
                if chand > new_sl:
                    new_sl = chand
            else:
                chand = cur + ATR_TRAIL_MULT * a
                if new_sl == 0 or chand < new_sl:
                    new_sl = chand
            st["stage"] = max(stage, 3)
    improve = ((pos.type == mt5.ORDER_TYPE_BUY and new_sl > pos.sl) or
               (pos.type == mt5.ORDER_TYPE_SELL and (pos.sl == 0 or new_sl < pos.sl)))
    if improve and modify_sl(pos, new_sl):
        self.log(f"    {pos.symbol} #{pos.ticket} SL trail: {pos.sl:.5f}->{new_sl:.5f} R={profit_r:.2f}")

# Cleanup state for closed
pending_tickets = {str(o.ticket) for o in (mt5.orders_get() or []) if o.magic == MAGIC_NUMBER}
alive = set(pos_tickets.keys()) | pending_tickets
for tk in list(self.state.keys()):
    if tk not in alive:
        # Trade closed - log it
        st = self.state[tk]
        deals = mt5.history_deals_get(position=int(tk))
        pnl = 0.0
        if deals:
            pnl = sum(d.profit + d.swap + d.commission for d in deals)
        append_trade_log({
            "ticket": tk, "symbol": st.get("symbol"), "side": st.get("side"),
            "entry": st.get("entry"), "sl": st.get("initial_sl"),
            "tp": st.get("tp"), "r_distance": st.get("r_distance"),
            "pnl": pnl, "closed_at": datetime.now().isoformat(),
        })
        self.state.pop(tk, None)
save_state(self.state)

```

```

def run(self):
    self.running = True
    while self.running:
        try:
            scan = {}
            for sym in self.symbols:
                scan[sym] = self.try_setup(sym)
            self.last_scan_info = scan
            self.manage_open()
        except Exception as e:
            self.log(f"[!] loop error: {e}")
            time.sleep(SCAN_INTERVAL_SEC)

def start(self):
    self.thread = threading.Thread(target=self.run, daemon=True)
    self.thread.start()

def stop(self):
    self.running = False

# =====
# Stats helpers
# =====
def calc_winrate():
    log = load_trade_log()
    if not log:
        return 0.0, 0, 0
    wins = sum(1 for t in log if (t.get("pnl") or 0) > 0)
    losses = sum(1 for t in log if (t.get("pnl") or 0) <= 0)
    total = wins + losses
    rate = (wins / total * 100) if total else 0.0
    return rate, wins, losses

def calc_total_pnl():
    return sum((t.get("pnl") or 0) for t in load_trade_log())

# =====
# Tkinter Dashboard
# =====
class Dashboard:
    REFRESH_MS = 2000

    def __init__(self, root, engine):
        self.root = root
        self.engine = engine
        self.root.title("Earn Bot - Live Dashboard")
        self.root.geometry("1200x720")
        self.root.configure(bg="#0e1117")

        style = ttk.Style()
        try:
            style.theme_use("clam")
        except Exception:
            pass
        style.configure("Treeview", background="#1a1f2b", foreground="#e6edf3",
                        fieldbackground="#1a1f2b", rowheight=24)
        style.configure("Treeview.Heading", background="#21262d", foreground="#58a6ff",
                        font=("Segoe UI", 9, "bold"))
        style.map("Treeview", background=[("selected", "#264f78")])

        # Header
        self.header = tk.Frame(root, bg="#161b22", pady=8)
        self.header.pack(fill="x")
        self.lbl_acc = tk.Label(self.header, text="Connecting...", fg="#58a6ff",
                                bg="#161b22", font=("Segoe UI", 11, "bold"))
        self.lbl_acc.pack(side="left", padx=15)
        self.lbl_stats = tk.Label(self.header, text="", fg="#7ee787",
                                bg="#161b22", font=("Segoe UI", 10))
        self.lbl_stats.pack(side="left", padx=15)
        self.lbl_kz = tk.Label(self.header, text="", fg="#f0883e",
                                bg="#161b22", font=("Segoe UI", 10))
        self.lbl_kz.pack(side="right", padx=15)

```

```

# Next setup forecast bar
self.next_bar = tk.Frame(root, bg="#1a2332", pady=6)
self.next_bar.pack(fill="x")
tk.Label(self.next_bar, text="NEXT SETUP:", fg="#f0883e",
         bg="#1a2332", font=("Segoe UI", 10, "bold")).pack(side="left", padx=15)
self.lbl_next = tk.Label(self.next_bar, text="scanning...", fg="#e6edf3",
                        bg="#1a2332", font=("Segoe UI", 10))
self.lbl_next.pack(side="left", padx=5)
self.lbl_next2 = tk.Label(self.next_bar, text="", fg="#58a6ff",
                        bg="#1a2332", font=("Segoe UI", 9))
self.lbl_next2.pack(side="left", padx=15)

# Notebook
self.nb = ttk.Notebook(root)
self.nb.pack(fill="both", expand=True, padx=10, pady=10)

# Tab 1 - Active trades
self.tab_trades = tk.Frame(self.nb, bg="#0e1117")
self.nb.add(self.tab_trades, text=" Active Trades ")
cols = ("ticket", "symbol", "side", "lot", "entry", "sl", "tp", "rr_init",
        "current", "r_now", "pnl", "stage")
self.tv_trades = ttk.Treeview(self.tab_trades, columns=cols, show="headings", height=15)
widths = {"ticket": 80, "symbol": 80, "side": 60, "lot": 60, "entry": 90, "sl": 90,
          "tp": 90, "rr_init": 70, "current": 90, "r_now": 70, "pnl": 80, "stage": 80}
for c in cols:
    self.tv_trades.heading(c, text=c.upper())
    self.tv_trades.column(c, width=widths[c], anchor="center")
self.tv_trades.pack(fill="both", expand=True, padx=8, pady=8)

# Tab 2 - Setup scanner
self.tab_scan = tk.Frame(self.nb, bg="#0e1117")
self.nb.add(self.tab_scan, text=" Setup Scanner ")
scols = ("symbol", "bias", "sweep", "choch", "fvg", "progress", "rr", "spread", "status")
self.tv_scan = ttk.Treeview(self.tab_scan, columns=scols, show="headings", height=15)
widths_s = {"symbol": 80, "bias": 60, "sweep": 60, "choch": 60, "fvg": 60,
            "progress": 80, "rr": 60, "spread": 70, "status": 200}
for c in scols:
    self.tv_scan.heading(c, text=c.upper())
    self.tv_scan.column(c, width=widths_s.get(c, 100), anchor="center")
self.tv_scan.pack(fill="both", expand=True, padx=8, pady=8)

# Tab 3 - Closed trades / win-rate
self.tab_log = tk.Frame(self.nb, bg="#0e1117")
self.nb.add(self.tab_log, text=" Closed Trades ")
lcols = ("closed_at", "symbol", "side", "entry", "sl", "tp", "pnl", "result")
self.tv_log = ttk.Treeview(self.tab_log, columns=lcols, show="headings", height=15)
lwidths = {"closed_at": 150, "symbol": 80, "side": 60, "entry": 90, "sl": 90,
            "tp": 90, "pnl": 90, "result": 80}
for c in lcols:
    self.tv_log.heading(c, text=c.upper())
    self.tv_log.column(c, width=lwidths[c], anchor="center")
self.tv_log.pack(fill="both", expand=True, padx=8, pady=8)

# Tab 4 - Chart
self.tab_chart = tk.Frame(self.nb, bg="#0e1117")
self.nb.add(self.tab_chart, text=" Chart View ")
ctrl = tk.Frame(self.tab_chart, bg="#0e1117")
ctrl.pack(fill="x", padx=8, pady=4)
tk.Label(ctrl, text="Symbol:", fg="#e6edf3", bg="#0e1117").pack(side="left", padx=6)
self.sym_var = tk.StringVar()
self.cb_sym = ttk.Combobox(ctrl, textvariable=self.sym_var, width=15)
self.cb_sym.pack(side="left", padx=4)
tk.Button(ctrl, text="Draw Chart", command=self.draw_chart,
         bg="#238636", fg="white", font=("Segoe UI", 9, "bold"),
         relief="flat", padx=14).pack(side="left", padx=10)
if not HAS_MPL:
    tk.Label(ctrl, text="(install matplotlib for charts)",
            fg="#f85149", bg="#0e1117").pack(side="left", padx=10)
self.chart_frame = tk.Frame(self.tab_chart, bg="#0e1117")
self.chart_frame.pack(fill="both", expand=True, padx=8, pady=8)
self.canvas = None

# Tab 5 - Log
self.tab_console = tk.Frame(self.nb, bg="#0e1117")

```

```

self.nb.add(self.tab_console, text=" Console Log ")
self.txt = tk.Text(self.tab_console, bg="#0d1117", fg="#7ee787",
                    insertbackground="white", font=("Consolas", 9),
                    wrap="word")
self.txt.pack(fill="both", expand=True, padx=8, pady=8)

# Hook engine logs to the text widget
self.engine.log_callback = self.append_log
self.log_q = queue.Queue()

# Start refresh
self.root.after(self.REFRESH_MS, self.refresh)
self.root.protocol("WM_DELETE_WINDOW", self.on_close)

def append_log(self, msg):
    # Called from background thread; route via queue
    self.log_q.put(msg)

def drain_logs(self):
    while True:
        try:
            msg = self.log_q.get_nowait()
        except queue.Empty:
            break
        self.txt.insert("end", msg + "\n")
        self.txt.see("end")

def refresh(self):
    try:
        self.drain_logs()
        self.update_header()
        self.update_trades()
        self.update_scan()
        self.update_log()
    except Exception as e:
        print(f"refresh err: {e}")
    self.root.after(self.REFRESH_MS, self.refresh)

def update_header(self):
    acc = mt5.account_info()
    if acc:
        self.lbl_acc.config(
            text=f"Acc {acc.login} | {acc.server} | "
                f"Bal: {acc.balance:.2f} | Eq: {acc.equity:.2f} {acc.currency}"
        )
    rate, wins, losses = calc_winrate()
    pnl = calc_total_pnl()
    col = "#7ee787" if pnl >= 0 else "#f85149"
    self.lbl_stats.config(
        text=f"WinRate {rate:.1f}% ({wins}W / {losses}L) | Total P&L: {pnl:+.2f}",
        fg=col,
    )
    self.lbl_kz.config(
        text=f"UTC {datetime.now(timezone.utc):%H:%M} Killzone: "
            f"'YES' if in_killzone() else 'no'"
    )

def update_trades(self):
    for r in self.tv_trades.get_children():
        self.tv_trades.delete(r)
    positions = mt5.positions_get() or []
    for pos in positions:
        if pos.magic != MAGIC_NUMBER:
            continue
        st = self.engine.state.get(str(pos.ticket), {})
        rd = st.get("r_distance", 0)
        tick = mt5.symbol_info_tick(pos.symbol)
        cur = tick.bid if (tick and pos.type == mt5.ORDER_TYPE_BUY) else (tick.ask if tick else 0)
        entry = st.get("entry", pos.price_open)
        r_now = 0
        if rd > 0 and cur:
            r_now = (cur - entry) / rd if pos.type == mt5.ORDER_TYPE_BUY else (entry - cur) / rd
        rr_init = (abs(pos.tp - entry) / rd) if rd > 0 and pos.tp else 0
        stage_names = {0: "INIT", 1: "BE", 2: "+0.5R", 3: "ATR-TRAIL"}
        stg = stage_names.get(st.get("stage", 0), "?")

```

```

        side = "BUY" if pos.type == mt5.ORDER_TYPE_BUY else "SELL"
        tag = "win" if pos.profit >= 0 else "loss"
        self.tv_trades.insert("", "end", values=(
            pos.ticket, pos.symbol, side, pos.volume,
            f"{entry:.5f}", f"{pos.sl:.5f}", f"{pos.tp:.5f}",
            f"{rr_init:.2f}", f"{cur:.5f}",
            f"{r_now:+.2f}R", f"{pos.profit:+.2f}", stg
        ), tags=(tag,))
    self.tv_trades.tag_configure("win", foreground="#7ee787")
    self.tv_trades.tag_configure("loss", foreground="#f85149")

# Also include pending limits (italic-ish via prefix)
orders = mt5.orders_get() or []
for o in orders:
    if o.magic != MAGIC_NUMBER:
        continue
    st = self.engine.state.get(str(o.ticket), {})
    rd = st.get("r_distance", 0)
    entry = st.get("entry", o.price_open)
    rr_init = (abs(o.tp - entry) / rd) if rd > 0 and o.tp else 0
    side = "BUY" if o.type in (mt5.ORDER_TYPE_BUY_LIMIT, mt5.ORDER_TYPE_BUY) else "SELL"
    self.tv_trades.insert("", "end", values=(
        f"P{o.ticket}", o.symbol, side, o.volume_initial,
        f"{entry:.5f}", f"{o.sl:.5f}", f"{o.tp:.5f}",
        f"{rr_init:.2f}", "PENDING", "-", "-", "WAITING"
    ), tags=("pending",))
    self.tv_trades.tag_configure("pending", foreground="#f0883e")

def update_scan(self):
    for r in self.tv_scan.get_children():
        self.tv_scan.delete(r)
    scan = self.engine.last_scan_info

# Build a list with progress score so we can sort + forecast next setup
rows = []
for sym in self.engine.symbols:
    info = scan.get(sym, {})
    score = 0
    if info.get("bias"):
        score += 1
    if info.get("sweep"):
        score += 1
    if info.get("choch"):
        score += 1
    if info.get("fvg"):
        score += 1
    if info.get("fvg") and info.get("rr"):
        status, tag = "READY -> FIRING", "ready"
        next_step = "executing"
    elif info.get("choch"):
        status, tag = "3/4 - need FVG", "watch"
        next_step = "waiting for FVG retrace"
    elif info.get("sweep"):
        status, tag = "2/4 - need CHOCH", "watch"
        next_step = "waiting for CHOCH on M5"
    elif info.get("bias"):
        status, tag = "1/4 - need sweep", "neutral"
        next_step = "waiting for liquidity sweep"
    else:
        status, tag = "0/4 - no bias", "idle"
        next_step = "no clear H1 trend"
    rows.append((score, sym, info, status, tag, next_step))

# Sort by progress descending so closest-to-fire pairs appear at top
rows.sort(key=lambda r: -r[0])

for score, sym, info, status, tag, next_step in rows:
    bias = info.get("bias") or "-"
    sweep = "YES" if info.get("sweep") else "no"
    choch = "YES" if info.get("choch") else "no"
    fvg = "YES" if info.get("fvg") else "no"
    rr = info.get("rr") or "-"
    sp = spread_points(sym)
    self.tv_scan.insert("", "end",
        values=(sym, bias, sweep, choch, fvg, f"{score}/4", rr, sp, status),

```

```

        tags=(tag,))
self.tv_scan.tag_configure("ready", foreground="#7ee787")
self.tv_scan.tag_configure("watch", foreground="#58a6ff")
self.tv_scan.tag_configure("neutral", foreground="#e6edf3")
self.tv_scan.tag_configure("idle", foreground="#6e7681")

# Update the "Next Setup" bar with the most-progressed pair
if rows:
    top_score, top_sym, top_info, top_status, _, top_next = rows[0]
    if top_score == 0:
        self.lbl_next.config(text="all pairs idle - waiting for clear H1 trend",
                             fg="#6e7681")
        self.lbl_next2.config(text="")
    else:
        bias = top_info.get("bias") or "-"
        arrow = "BUY" if bias == "UP" else "SELL"
        col = "#7ee787" if top_score == 4 else "#58a6ff"
        self.lbl_next.config(
            text=f"{top_sym} {arrow} ({top_score}/4) - {top_status}",
            fg=col,
        )
        self.lbl_next2.config(text=f"-> {top_next}")

def update_log(self):
    for r in self.tv_log.get_children():
        self.tv_log.delete(r)
    log = load_trade_log()
    for t in reversed(log[-100:]):
        pnl = t.get("pnl") or 0
        res = "WIN" if pnl > 0 else "LOSS"
        tag = "win" if pnl > 0 else "loss"
        self.tv_log.insert("", "end", values=(
            t.get("closed_at", "")[:19], t.get("symbol", ""), t.get("side", ""),
            f"{t.get('entry', 0):.5f}", f"{t.get('sl', 0):.5f}", f"{t.get('tp', 0):.5f}",
            f"{pnl:+.2f}", res
        ), tags=(tag,))
    self.tv_log.tag_configure("win", foreground="#7ee787")
    self.tv_log.tag_configure("loss", foreground="#f85149")

# Refresh combobox
if self.cb_sym["values"] != tuple(self.engine.symbols):
    self.cb_sym["values"] = self.engine.symbols
    if not self.sym_var.get() and self.engine.symbols:
        self.sym_var.set(self.engine.symbols[0])

def draw_chart(self):
    if not HAS_MPL:
        return
    sym = self.sym_var.get()
    if not sym:
        return
    df = get_rates(sym, "M15", 100)
    if df is None:
        return

    fig = Figure(figsize=(12, 6.5), facecolor="#0d1117")
    ax = fig.add_subplot(111, facecolor="#0d1117")

    # OHLC bars (manual)
    for i, row in df.iterrows():
        color = "#7ee787" if row["close"] >= row["open"] else "#f85149"
        # wick
        ax.plot([i, i], [row["low"], row["high"]], color=color, linewidth=0.8, zorder=2)
        # body
        body_low = min(row["open"], row["close"])
        body_high = max(row["open"], row["close"])
        ax.add_patch(Rectangle((i - 0.35, body_low), 0.7, max(body_high - body_low, 1e-9),
                              facecolor=color, edgecolor=color, zorder=3))

    # Direction
    bias = htf_bias(sym)
    ax.text(0.01, 0.97, f"{sym} M15 bias={bias or 'NONE'}",
           transform=ax.transAxes, fontsize=12, color="#58a6ff",
           fontweight="bold", verticalalignment="top")

```

```

# Sweep
if bias:
    sweep = detect_sweep(df, bias)
    if sweep:
        lvl = sweep["swept_level"]
        ax.axhline(lvl, color="#f0883e", linestyle="--", linewidth=1.2, alpha=0.8)
        ax.text(len(df) - 1, lvl, " SWEEP", color="#f0883e", fontsize=9,
                verticalalignment="center")

# FVG (on M5 -> show on M15 chart roughly via M5 data)
df_m5 = get_rates(sym, "M5", 200)
if df_m5 is not None and bias:
    sweep_idx_m5 = max(0, len(df_m5) - 30)
    fvg = find_fvg(df_m5, bias, sweep_idx_m5)
    if fvg:
        color = "#58a6ff" if bias == "UP" else "#f85149"
        ax.hspan(fvg["bot"], fvg["top"], facecolor=color, alpha=0.18, zorder=1)
        ax.axhline(fvg["mid"], color=color, linestyle=":", linewidth=1, alpha=0.7)
        ax.text(len(df) - 1, fvg["mid"], f" FVG mid", color=color, fontsize=9,
                verticalalignment="center")

# Open positions/orders for this symbol
for pos in (mt5.positions_get(symbol=sym) or []):
    if pos.magic != MAGIC_NUMBER:
        continue
    ax.axhline(pos.price_open, color="#58a6ff", linestyle="-", linewidth=1, alpha=0.9)
    ax.axhline(pos.sl, color="#f85149", linestyle="-", linewidth=1, alpha=0.9)
    ax.axhline(pos.tp, color="#7ee787", linestyle="-", linewidth=1, alpha=0.9)
    side = "BUY" if pos.type == mt5.ORDER_TYPE_BUY else "SELL"
    ax.text(0, pos.price_open, f" ENTRY ({side})", color="#58a6ff", fontsize=9,
            verticalalignment="center")
    ax.text(0, pos.sl, " SL", color="#f85149", fontsize=9, verticalalignment="center")
    ax.text(0, pos.tp, " TP", color="#7ee787", fontsize=9, verticalalignment="center")

ax.set_xlim(-1, len(df))
ax.tick_params(colors="#8b949e")
for spine in ax.spines.values():
    spine.set_color("#30363d")
ax.grid(True, alpha=0.15)
fig.tight_layout()

for w in self.chart_frame.wininfo_children():
    w.destroy()
self.canvas = FigureCanvasTkAgg(fig, master=self.chart_frame)
self.canvas.draw()
self.canvas.get_tk_widget().pack(fill="both", expand=True)

def on_close(self):
    self.engine.stop()
    time.sleep(0.3)
    try:
        mt5.shutdown()
    except Exception:
        pass
    self.root.destroy()

# =====
# Main
# =====
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--no-gui", action="store_true", help="run without dashboard")
    args = parser.parse_args()

    print("=" * 70)
    print(" EARN BOT - SMC/ICT MT5 Trading Bot with Dashboard")
    print("=" * 70)

    acc, status = mt5_connect()
    if acc is None:
        print(f"[!] Login failed: {status}")
        sys.exit(1)
    print(f"[+] Logged in: {acc.login} | Balance: {acc.balance:.2f} {acc.currency}")
    print(f" Server: {acc.server} | Leverage: 1:{acc.leverage}")

```

```
print(f"[*] Risk per trade: {RISK_PERCENT}% | Max trades: {MAX_OPEN_TRADES}")
print(f"[*] Killzones (UTC): London {LONDON_KZ}, NY {NEWYORK_KZ}")
print(f"[*] In killzone now: {in_killzone()}")
print()

engine = BotEngine()
engine.resolve_symbols()
if not engine.symbols:
    print("[!] No tradable symbols.")
    mt5.shutdown()
    sys.exit(1)

engine.start()

if args.no_gui or not HAS_TK:
    print("[*] Running headless. Ctrl+C to stop.")
    try:
        while True:
            time.sleep(60)
            rate, w, l = calc_winrate()
            pnl = calc_total_pnl()
            print(f"[stat] WinRate {rate:.1f}% ({w}W/{l}L) | P&L {pnl:+.2f} | "
                  f"Open {open_count()} | Pending {pending_count()}")
    except KeyboardInterrupt:
        print("\n[*] Stopping...")
    finally:
        engine.stop()
        mt5.shutdown()
else:
    root = tk.Tk()
    Dashboard(root, engine)
    root.mainloop()

if __name__ == "__main__":
    main()
```


Path in repo: /smc_ict_bot.py

[illegible]

```

# Killzones (UTC). Only forex trades in killzone; crypto trades 24/7.
LONDON_KZ = (7, 10)      # 07:00 - 10:00 UTC
NEWYORK_KZ = (12, 15)    # 12:00 - 15:00 UTC

# Structure / pivot detection
PIVOT_LOOKBACK_H1 = 5    # 5-bar fractal on H1 (slower, cleaner pivots)
PIVOT_LOOKBACK_M15 = 3   # 3-bar fractal on M15
SWEEP_LOOKBACK_M15 = 30  # bars to look back for stop-hunt level
ATR_PERIOD = 14
SL_BUFFER_ATR = 0.3      # extra distance below sweep for SL

# Entry params
PENDING_ORDER_EXPIRY_HOURS = 4 # cancel limit if not filled in 4h
MIN_RR_TO_TARGET = 1.5        # skip setup if next pool < 1.5R away

# Trailing (same as v2 final bot)
BREAKEVEN_AT_R = 1.0
LOCK_HALF_AT_R = 1.5
TRAIL_START_R = 2.0
ATR_TRAIL_MULT = 1.5

# Spread caps
MAX_SPREAD_FX = 25
MAX_SPREAD_CRYPTO = 5000

# Loop
SCAN_INTERVAL_SEC = 10      # SMC setups are slower; 10s is plenty
RUN_MINUTES = 0             # 0 = forever
STATE_FILE = "smc_bot_state.json"

# =====

# ----- Indicators -----
def atr(df: pd.DataFrame, p: int = 14) -> pd.Series:
    h, l, c = df["high"], df["low"], df["close"]
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    return tr.ewm(alpha=1/p, adjust=False).mean()

# ----- MT5 helpers -----
TF = {"M5": mt5.TIMEFRAME_M5, "M15": mt5.TIMEFRAME_M15,
      "H1": mt5.TIMEFRAME_H1, "H4": mt5.TIMEFRAME_H4}

def init_mt5() -> bool:
    kw = {"login": MT5_LOGIN, "password": MT5_PASSWORD, "server": MT5_SERVER}
    if MT5_PATH:
        kw["path"] = MT5_PATH
    if not mt5.initialize(**kw):
        print(f"[!] MT5 init failed: {mt5.last_error()}")
        return False
    acc = mt5.account_info()
    if acc is None:
        print(f"[!] Login failed: {mt5.last_error()}")
        return False
    print(f"[+] Logged in: {acc.login} | Bal: {acc.balance:.2f} {acc.currency} | "
          f"Server: {acc.server}")
    return True

def find_symbol(base: str) -> Optional[str]:
    for cand in [base, base + "m", base + "r", base + "_", base + "!", base + ".s"]:
        if mt5.symbol_info(cand) is not None:
            mt5.symbol_select(cand, True)
            return cand
    matches = mt5.symbols_get(f"*{base}*")
    if matches:
        mt5.symbol_select(matches[0].name, True)
        return matches[0].name
    return None

def get_rates(sym: str, tf_str: str, n: int = 300) -> Optional[pd.DataFrame]:

```

```

bars = mt5.copy_rates_from_pos(sym, TF[tf_str], 0, n)
if bars is None or len(bars) < 50:
    return None
df = pd.DataFrame(bars)
df["time"] = pd.to_datetime(df["time"], unit="s")
return df

def is_crypto(sym: str) -> bool:
    s = sym.upper()
    return any(c in s for c in ["BTC", "ETH", "XRP", "LTC", "DOGE", "SOL"])

def in_killzone() -> bool:
    h = datetime.now(timezone.utc).hour
    return (LONDON_KZ[0] <= h < LONDON_KZ[1]) or (NEWYORK_KZ[0] <= h < NEWYORK_KZ[1])

def spread_points(sym: str) -> int:
    info = mt5.symbol_info(sym)
    return info.spread if info else 9999

# =====
# SMC / ICT CORE ANALYSIS
# =====

def mark_pivots(df: pd.DataFrame, lb: int) -> pd.DataFrame:
    """Mark swing highs (PH) and swing lows (PL) using N-bar fractal."""
    df = df.copy()
    df["ph"] = False
    df["pl"] = False
    for i in range(lb, len(df) - lb):
        win_h = df["high"].iloc[i - lb:i + lb + 1]
        win_l = df["low"].iloc[i - lb:i + lb + 1]
        if df["high"].iloc[i] == win_h.max() and (win_h == df["high"].iloc[i]).sum() == 1:
            df.iat[i, df.columns.get_loc("ph")] = True
        if df["low"].iloc[i] == win_l.min() and (win_l == df["low"].iloc[i]).sum() == 1:
            df.iat[i, df.columns.get_loc("pl")] = True
    return df

def htf_bias(sym: str) -> Optional[str]:
    """
    H1 market structure bias.
    Look at last 4 confirmed pivots:
        HH + HL sequence => UP
        LH + LL sequence => DOWN
    """
    df = get_rates(sym, "H1", 250)
    if df is None:
        return None
    df = mark_pivots(df, PIVOT_LOOKBACK_H1)
    phs = df[df["ph"]].tail(3)["high"].values
    pls = df[df["pl"]].tail(3)["low"].values
    if len(phs) < 2 or len(pls) < 2:
        return None
    if phs[-1] > phs[-2] and pls[-1] > pls[-2]:
        return "UP"
    if phs[-1] < phs[-2] and pls[-1] < pls[-2]:
        return "DOWN"
    return None

def detect_sweep(df_m15: pd.DataFrame, direction: str) -> Optional[dict]:
    """
    Detect liquidity sweep on M15.
    For LONG (direction=UP): wick below recent swing low + close back inside.
    For SHORT (direction=DOWN): wick above recent swing high + close back inside.
    """
    last = df_m15.iloc[-2] # last closed candle
    window = df_m15.iloc[-(SWEEP_LOOKBACK_M15 + 2):-2]
    if len(window) < 10:
        return None

```

```

    if direction == "UP":
        prior_low = window["low"].min()
        if last["low"] < prior_low and last["close"] > prior_low:
            return {"swept_level": prior_low, "sweep_low": last["low"]}
    else:
        prior_high = window["high"].max()
        if last["high"] > prior_high and last["close"] < prior_high:
            return {"swept_level": prior_high, "sweep_high": last["high"]}
    return None

def find_fvg_after_sweep(df_m5: pd.DataFrame, direction: str,
                        min_idx: int = 0) -> Optional[dict]:
    """
    Find the most recent unfilled Fair Value Gap on M5 in trade direction.
    Bullish FVG: candle[i-2].high < candle[i].low (gap between)
    Bearish FVG: candle[i-2].low > candle[i].high
    """
    fvg = []
    for i in range(max(2, min_idx), len(df_m5)):
        c1 = df_m5.iloc[i - 2]
        c3 = df_m5.iloc[i]
        if direction == "UP" and c1["high"] < c3["low"]:
            fvg.append({"type": "BULL", "top": c3["low"], "bot": c1["high"],
                        "mid": (c3["low"] + c1["high"]) / 2, "idx": i})
        if direction == "DOWN" and c1["low"] > c3["high"]:
            fvg.append({"type": "BEAR", "top": c1["low"], "bot": c3["high"],
                        "mid": (c1["low"] + c3["high"]) / 2, "idx": i})

    if not fvg:
        return None

    # Take the most recent FVG that hasn't been filled (price hasn't fully passed thru)
    latest_close = df_m5.iloc[-1]["close"]
    for fvg in reversed(fvg):
        if direction == "UP":
            # Bullish FVG still valid if current close above the gap top (not yet retraced)
            if latest_close > fvg["top"]:
                return fvg
        else:
            if latest_close < fvg["bot"]:
                return fvg
    return None

def detect_choch_m5(df_m5: pd.DataFrame, direction: str, sweep_idx: int) -> bool:
    """
    Change of Character on M5 after the sweep:
    For UP: price closed above the most recent M5 swing high formed AFTER the sweep low.
    For DOWN: vice versa.
    """
    pdf = mark_pivots(df_m5, PIVOT_LOOKBACK_M15)
    after_sweep = pdf.iloc[sweep_idx:]
    if direction == "UP":
        recent_ph = after_sweep[after_sweep["ph"]].tail(1)
        if recent_ph.empty:
            return False
        ph_price = recent_ph["high"].iloc[0]
        return df_m5.iloc[-1]["close"] > ph_price
    else:
        recent_pl = after_sweep[after_sweep["pl"]].tail(1)
        if recent_pl.empty:
            return False
        pl_price = recent_pl["low"].iloc[0]
        return df_m5.iloc[-1]["close"] < pl_price

def find_target_liquidity(df_h1: pd.DataFrame, direction: str,
                        entry: float) -> Optional[float]:
    """Next opposing liquidity pool (next swing high for longs / low for shorts)."""
    pdf = mark_pivots(df_h1, PIVOT_LOOKBACK_H1)
    if direction == "UP":
        future_highs = pdf[pdf["ph"] & (pdf["high"] > entry)]["high"]
        if not future_highs.empty:
            return float(future_highs.min())

```

```

        return float(pdf["high"].tail(50).max())
    else:
        future_lows = pdf[pdf["pl"] & (pdf["low"] < entry)]["low"]
        if not future_lows.empty:
            return float(future_lows.max())
        return float(pdf["low"].tail(50).min())

# =====
# POSITION SIZING + ORDERS
# =====

def calc_lot(sym: str, entry: float, sl: float, risk_pct: float) -> float:
    acc = mt5.account_info()
    info = mt5.symbol_info(sym)
    if acc is None or info is None:
        return 0.0
    risk_money = acc.balance * (risk_pct / 100.0)
    sl_dist = abs(entry - sl)
    if sl_dist <= 0:
        return 0.0
    tick_size = info.trade_tick_size or info.point
    tick_value = info.trade_tick_value
    if not tick_size or not tick_value:
        return 0.0
    money_per_lot = (sl_dist / tick_size) * tick_value
    if money_per_lot <= 0:
        return 0.0
    lot = risk_money / money_per_lot
    step = info.volume_step or 0.01
    lot = round(lot / step) * step
    lot = max(info.volume_min, min(info.volume_max, lot))
    return round(lot, 2)

def place_pending_limit(sym: str, side: str, lot: float, price: float,
                        sl: float, tp: float, comment: str) -> Optional[int]:
    """Pending limit at FVG midpoint - fills only if price retraces."""
    info = mt5.symbol_info(sym)
    if info is None or lot <= 0:
        return None
    digits = info.digits
    otype = mt5.ORDER_TYPE_BUY_LIMIT if side == "BUY" else mt5.ORDER_TYPE_SELL_LIMIT
    expiry = int((datetime.now() + timedelta(hours=PENDING_ORDER_EXPIRY_HOURS)).timestamp())
    req = {
        "action": mt5.TRADE_ACTION_PENDING,
        "symbol": sym, "volume": lot, "type": otype,
        "price": round(price, digits),
        "sl": round(sl, digits),
        "tp": round(tp, digits),
        "deviation": 30,
        "magic": MAGIC_NUMBER,
        "comment": comment[:31],
        "type_time": mt5.ORDER_TIME_SPECIFIED,
        "expiration": expiry,
        "type_filling": mt5.ORDER_FILLING_RETURN,
    }
    r = mt5.order_send(req)
    if r is None or r.retcode != mt5.TRADE_RETCODE_DONE:
        msg = getattr(r, "comment", "") if r else mt5.last_error()
        print(f"[!] {sym} {side} LIMIT failed: {msg}")
        return None
    print(f"[+] {sym} {side}_LIMIT {lot} @ {price:.5f} "
          f"SL={sl:.5f} TP={tp:.5f} ({comment})")
    return r.order

def modify_sl(pos, new_sl: float) -> bool:
    info = mt5.symbol_info(pos.symbol)
    digits = info.digits if info else 5
    req = {
        "action": mt5.TRADE_ACTION_SLTP,
        "position": pos.ticket,
        "symbol": pos.symbol,
        "sl": round(new_sl, digits),
    }

```

```

        "tp": round(pos.tp, digits),
    }
    r = mt5.order_send(req)
    return r is not None and r.retcode == mt5.TRADE_RETCODE_DONE

# ----- State persistence -----
def load_state() -> dict:
    if os.path.exists(STATE_FILE):
        try:
            with open(STATE_FILE) as f:
                return json.load(f)
        except Exception:
            pass
    return {}

def save_state(s: dict):
    try:
        with open(STATE_FILE, "w") as f:
            json.dump(s, f)
    except Exception as e:
        print(f"[!] state save failed: {e}")

def open_count(sym: Optional[str] = None) -> int:
    pos = mt5.positions_get(symbol=sym) if sym else mt5.positions_get()
    if pos is None:
        return 0
    return sum(1 for p in pos if p.magic == MAGIC_NUMBER)

def pending_count(sym: Optional[str] = None) -> int:
    orders = mt5.orders_get(symbol=sym) if sym else mt5.orders_get()
    if orders is None:
        return 0
    return sum(1 for o in orders if o.magic == MAGIC_NUMBER)

# =====
# ENTRY PIPELINE - SMC/ICT confluence check
# =====

def try_smc_setup(sym: str, state: dict) -> bool:
    # Caps
    if open_count() + pending_count() >= MAX_OPEN_TRADES:
        return False
    if open_count(sym) + pending_count(sym) >= MAX_TRADES_PER_PAIR:
        return False

    # Killzone filter - only forex; crypto 24/7
    if not is_crypto(sym) and not in_killzone():
        return False

    # Spread filter
    cap = MAX_SPREAD_CRYPTO if is_crypto(sym) else MAX_SPREAD_FX
    if spread_points(sym) > cap:
        return False

    # 1. HTF bias (H1)
    direction = htf_bias(sym)
    if direction is None:
        return False

    # 2. Liquidity sweep on M15
    df_m15 = get_rates(sym, "M15", 200)
    if df_m15 is None:
        return False
    sweep = detect_sweep(df_m15, direction)
    if sweep is None:
        return False

    # 3. CHOCH on M5
    df_m5 = get_rates(sym, "M5", 200)
    if df_m5 is None:

```

```

        return False
    # Approximate sweep index on M5: last 1-2 hours of M5 = ~24 candles
    sweep_idx_m5 = max(0, len(df_m5) - 30)
    if not detect_choch_m5(df_m5, direction, sweep_idx_m5):
        return False

    # 4. Find unfilled FVG on M5 in trade direction (POI for entry)
    fvg = find_fvg_after_sweep(df_m5, direction, min_idx=sweep_idx_m5)
    if fvg is None:
        return False

    # 5. Build entry/SL/TP
    atr_m15 = atr(df_m15, ATR_PERIOD).iloc[-1]
    if pd.isna(atr_m15):
        return False

    entry = fvg["mid"]                                # enter at FVG midpoint via limit
    if direction == "UP":
        sl = sweep["sweep_low"] - SL_BUFFER_ATR * atr_m15
    else:
        sl = sweep["sweep_high"] + SL_BUFFER_ATR * atr_m15

    risk_dist = abs(entry - sl)
    if risk_dist <= 0:
        return False

    # 6. Find target liquidity
    df_h1 = get_rates(sym, "H1", 200)
    target = find_target_liquidity(df_h1, direction, entry) if df_h1 is not None else None
    if target is None:
        # fallback: 2R
        target = entry + 2 * risk_dist if direction == "UP" else entry - 2 * risk_dist

    rr = abs(target - entry) / risk_dist
    if rr < MIN_RR_TO_TARGET:
        print(f"[~] {sym} setup found but RR={rr:.2f} too low; skip")
        return False

    # 7. Place pending limit at FVG midpoint
    side = "BUY" if direction == "UP" else "SELL"
    lot = calc_lot(sym, entry, sl, RISK_PERCENT)
    if lot <= 0:
        print(f"[!] {sym} lot=0 - check tick value")
        return False

    print(f"[*] SMC setup: {sym} {side} | bias={direction} | sweep@{sweep['swept_level']:.5f} "
          f"| FVG entry={entry:.5f} | SL={sl:.5f} | TP={target:.5f} | RR={rr:.2f}")

    ticket = place_pending_limit(sym, side, lot, entry, sl, target,
                                 f"SMC_{direction}_RR{rr:.1f}")

    if ticket is None:
        return False

    state[str(ticket)] = {
        "type": "pending",
        "symbol": sym, "side": side,
        "entry": entry, "initial_sl": sl, "tp": target,
        "r_distance": risk_dist, "stage": 0,
    }
    save_state(state)
    return True

def scan_all(state: dict, symbols: list) -> int:
    placed = 0
    for sym in symbols:
        try:
            if try_smc_setup(sym, state):
                placed += 1
        except Exception as e:
            print(f"[!] {sym} scan error: {e}")
    return placed

# =====

```

```

# TRADE MANAGEMENT - promote pending->position, then smart trail
# =====

def manage_trades(state: dict):
    # Promote: when pending fills, MT5 creates a position with same ticket-or-deal
    positions = mt5.positions_get() or []

    # Build lookup of currently active tickets
    pos_tickets = {str(p.ticket): p for p in positions if p.magic == MAGIC_NUMBER}

    # Move state entries from "pending" -> "position" for filled orders
    for tk, st in list(state.items()):
        if st.get("type") == "pending" and tk in pos_tickets:
            st["type"] = "position"
            print(f"[+] Pending #{tk} filled - now active position")
            save_state(state)

    # Trail open positions
    for pos in positions:
        if pos.magic != MAGIC_NUMBER:
            continue
        st = state.get(str(pos.ticket))
        if st is None:
            continue
        rd = st.get("r_distance", 0)
        if rd <= 0:
            continue

        tick = mt5.symbol_info_tick(pos.symbol)
        if tick is None:
            continue
        cur = tick.bid if pos.type == mt5.ORDER_TYPE_BUY else tick.ask
        entry = st["entry"]

        if pos.type == mt5.ORDER_TYPE_BUY:
            profit_r = (cur - entry) / rd
        else:
            profit_r = (entry - cur) / rd

        if profit_r <= 0:
            continue

        new_sl = pos.sl
        stage = st.get("stage", 0)

        # +1R -> BE
        if profit_r >= BREAKEVEN_AT_R and stage < 1:
            new_sl = entry
            st["stage"] = 1
            print(f"[~] {pos.symbol} #{pos.ticket} -> BE @ {profit_r:.2f}R")

        # +1.5R -> lock half
        if profit_r >= LOCK_HALF_AT_R and stage < 2:
            new_sl = entry + 0.5 * rd if pos.type == mt5.ORDER_TYPE_BUY else entry - 0.5 * rd
            st["stage"] = 2
            print(f"[~] {pos.symbol} #{pos.ticket} -> LOCK +0.5R @ {profit_r:.2f}R")

        # +2R+ -> ATR Chandelier trail
        if profit_r >= TRAIL_START_R:
            df = get_rates(pos.symbol, "M15", 50)
            if df is not None:
                a = atr(df, ATR_PERIOD).iloc[-1]
                if pos.type == mt5.ORDER_TYPE_BUY:
                    chand = cur - ATR_TRAIL_MULT * a
                    if chand > new_sl:
                        new_sl = chand
                else:
                    chand = cur + ATR_TRAIL_MULT * a
                    if new_sl == 0 or chand < new_sl:
                        new_sl = chand
            st["stage"] = max(stage, 3)

    improve = (
        (pos.type == mt5.ORDER_TYPE_BUY and new_sl > pos.sl)
        or (pos.type == mt5.ORDER_TYPE_SELL and (pos.sl == 0 or new_sl < pos.sl))
    )

```



```

    )
    if improve and modify_sl(pos, new_sl):
        print(f"    SL: {pos.sl:.5f} -> {new_sl:.5f} (R={profit_r:.2f})")

# Cleanup: drop state for tickets that are neither open positions nor pending orders
pending_tickets = {str(o.ticket) for o in (mt5.orders_get() or [])
                    if o.magic == MAGIC_NUMBER}
alive = set(pos_tickets.keys()) | pending_tickets
for tk in list(state.keys()):
    if tk not in alive:
        state.pop(tk, None)
save_state(state)

# =====
#  MAIN
# =====
def main():
    if not init_mt5():
        return

    requested = SYMBOLS_FOREX + SYMBOLS_CRYPTO
    symbols = []
    for s in requested:
        actual = find_symbol(s)
        if actual:
            symbols.append(actual)
            print(f"    [+] {s} -> {actual}")
        else:
            print(f"    [-] {s} not available")

    if not symbols:
        print("[!] No symbols.")
        mt5.shutdown()
        return

    state = load_state()
    end = (datetime.now() + timedelta(minutes=RUN_MINUTES)) if RUN_MINUTES > 0 else None
    print(f"\n[*] SMC/ICT Bot live. Risk={RISK_PERCENT}% | Symbols={len(symbols)} | "
          f"'Forever' if end is None else f'Till {end:%H:%M:%S}')"
    print(f"[*] Killzones (UTC): London {LONDON_KZ[0]:02d}-{LONDON_KZ[1]:02d}, "
          f"NY {NEWYORK_KZ[0]:02d}-{NEWYORK_KZ[1]:02d} | "
          f"Now: {datetime.now(timezone.utc):%H:%M} UTC | "
          f"In KZ: {in_killzone()}\n")

    try:
        while True:
            if end and datetime.now() >= end:
                print("[*] Time up.")
                break
            placed = scan_all(state, symbols)
            manage_trades(state)
            if placed:
                print(f"[i] Placed {placed} | Open: {open_count()} | "
                      f"Pending: {pending_count()}")
                time.sleep(SCAN_INTERVAL_SEC)
    except KeyboardInterrupt:
        print("\n[*] Stopped by user.")
    finally:
        mt5.shutdown()
        print("[*] Disconnected.")

if __name__ == "__main__":
    main()

```

[illegible]

```

# Initial Risk:Reward target (fallback TP if trailing fails)
INITIAL_RR = 2.0

# Trailing thresholds (in R-multiples)
BREAKEVEN_AT_R = 1.0      # Move SL to entry when profit reaches 1R
LOCK_HALF_AT_R = 1.5      # Move SL to +0.5R when profit reaches 1.5R
TRAIL_START_R = 2.0       # Start ATR-trailing after this
ATR_TRAIL_MULT = 1.5      # Chandelier exit distance

# Trade caps - take every valid setup
MAX_OPEN_TRADES = 999      # effectively unlimited
MAX_TRADES_PER_PAIR = 1    # still 1 per symbol

# Higher TF trend filter
HTF_EMA_FAST = 50
HTF_EMA_SLOW = 200
ADX_MIN = 22

# Entry filters
ENTRY_EMA = 20             # M15 pullback EMA
RSI_PERIOD = 14
ATR_PERIOD = 14
SL_ATR_MULT = 1.5

# Loop
SCAN_INTERVAL_SEC = 5
RUN_MINUTES = 0           # 0 = forever
MAGIC_NUMBER = 777001

# Spread cap (in points). Skip if broker spread > this.
MAX_SPREAD_POINTS_FX = 25
MAX_SPREAD_POINTS_CRYPTO = 5000 # crypto has wider spreads

# State persistence (for trailing - survives restart)
STATE_FILE = "mt5_bot_state.json"

# =====

# ----- Indicators -----
def ema(s: pd.Series, p: int) -> pd.Series:
    return s.ewm(span=p, adjust=False).mean()

def rsi(s: pd.Series, p: int = 14) -> pd.Series:
    d = s.diff()
    g = d.clip(lower=0).ewm(alpha=1/p, adjust=False).mean()
    l = (-d.clip(upper=0)).ewm(alpha=1/p, adjust=False).mean()
    rs = g / l.replace(0, np.nan)
    return 100 - (100 / (1 + rs))

def atr(df: pd.DataFrame, p: int = 14) -> pd.Series:
    h, l, c = df["high"], df["low"], df["close"]
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    return tr.ewm(alpha=1/p, adjust=False).mean()

def adx(df: pd.DataFrame, p: int = 14) -> pd.Series:
    h, l, c = df["high"], df["low"], df["close"]
    plus_dm = (h.diff()).where(lambda x: x > 0, 0)
    minus_dm = (-l.diff()).where(lambda x: x > 0, 0)
    plus_dm = plus_dm.where(plus_dm > minus_dm, 0)
    minus_dm = minus_dm.where(minus_dm > plus_dm, 0)
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    atr_v = tr.ewm(alpha=1/p, adjust=False).mean()
    plus_di = 100 * plus_dm.ewm(alpha=1/p, adjust=False).mean() / atr_v.replace(0, np.nan)
    minus_di = 100 * minus_dm.ewm(alpha=1/p, adjust=False).mean() / atr_v.replace(0, np.nan)
    dx = 100 * (plus_di - minus_di).abs() / (plus_di + minus_di).replace(0, np.nan)
    return dx.ewm(alpha=1/p, adjust=False).mean()

# ----- MT5 helpers -----
TF = {"M5": mt5.TIMEFRAME_M5, "M15": mt5.TIMEFRAME_M15,

```

```

    "H1": mt5.TIMEFRAME_H1, "H4": mt5.TIMEFRAME_H4}

def init_mt5() -> bool:
    kw = {"login": MT5_LOGIN, "password": MT5_PASSWORD, "server": MT5_SERVER}
    if MT5_PATH:
        kw["path"] = MT5_PATH
    if not mt5.initialize(**kw):
        print(f"[!] MT5 init failed: {mt5.last_error()}")
        return False
    acc = mt5.account_info()
    if acc is None:
        print(f"[!] Login failed: {mt5.last_error()}")
        return False
    print(f"[+] Logged in: {acc.login} | Balance: {acc.balance:.2f} {acc.currency} | Server: {acc.server}")
    return True

def find_symbol(base: str) -> Optional[str]:
    """Auto-detect broker-specific symbol name (BTCUSD, BTCUSDM, BTCUSD.r, etc.)"""
    for cand in [base, base + "m", base + ".r", base + "_", base + "!", base + ".s"]:
        info = mt5.symbol_info(cand)
        if info is not None:
            mt5.symbol_select(cand, True)
            return cand
    matches = mt5.symbols_get(f"*{base}*")
    if matches:
        name = matches[0].name
        mt5.symbol_select(name, True)
        return name
    return None

def get_rates(sym: str, tf_str: str, n: int = 300) -> Optional[pd.DataFrame]:
    bars = mt5.copy_rates_from_pos(sym, TF[tf_str], 0, n)
    if bars is None or len(bars) < 50:
        return None
    df = pd.DataFrame(bars)
    df["time"] = pd.to_datetime(df["time"], unit="s")
    return df

def spread_points(sym: str) -> int:
    info = mt5.symbol_info(sym)
    return info.spread if info else 9999

def is_crypto(sym: str) -> bool:
    s = sym.upper()
    return any(c in s for c in ["BTC", "ETH", "XRP", "LTC", "BCH", "DOGE", "SOL"])

# ----- Multi-timeframe analysis -----
def htf_trend(sym: str) -> Optional[str]:
    """H4 macro trend: 'UP' / 'DOWN' / None"""
    df = get_rates(sym, "H4", 250)
    if df is None:
        return None
    df["ef"] = ema(df["close"], HTF_EMA_FAST)
    df["es"] = ema(df["close"], HTF_EMA_SLOW)
    last = df.iloc[-1]
    if last["close"] > last["ef"] > last["es"]:
        return "UP"
    if last["close"] < last["ef"] < last["es"]:
        return "DOWN"
    return None

def h1_strength(sym: str, direction: str) -> bool:
    """H1 confirms trend strength via ADX + EMA50 alignment"""
    df = get_rates(sym, "H1", 200)
    if df is None:
        return False
    df["ema50"] = ema(df["close"], 50)

```

```

    df["adx"] = adx(df, 14)
    last = df.iloc[-1]
    if pd.isna(last["adx"]) or last["adx"] < ADX_MIN:
        return False
    if direction == "UP":
        return last["close"] > last["ema50"]
    return last["close"] < last["ema50"]

def m15_entry(sym: str, direction: str):
    """M15 pullback + RSI cross. Returns (entry_price, sl_price, atr_value) or None"""
    df = get_rates(sym, "M15", 200)
    if df is None:
        return None
    df["e20"] = ema(df["close"], ENTRY_EMA)
    df["rsi"] = rsi(df["close"], RSI_PERIOD)
    df["atr"] = atr(df, ATR_PERIOD)
    a, b = df.iloc[-1], df.iloc[-2] # last closed, prev closed
    if pd.isna(a["atr"]) or pd.isna(a["rsi"]):
        return None

    atr_val = a["atr"]
    price = a["close"]

    if direction == "UP":
        # Pullback near EMA20, RSI crossed up through 50, bullish candle
        if (b["low"] <= b["e20"] * 1.001 # touched/near EMA20
            and b["rsi"] < 50 <= a["rsi"] # RSI crossed up
            and a["close"] > a["open"]): # bullish close
            sl = price - SL_ATR_MULT * atr_val
            return (price, sl, atr_val)

    if direction == "DOWN":
        if (b["high"] >= b["e20"] * 0.999
            and b["rsi"] > 50 >= a["rsi"]
            and a["close"] < a["open"]):
            sl = price + SL_ATR_MULT * atr_val
            return (price, sl, atr_val)

    return None

def m5_trigger(sym: str, direction: str) -> bool:
    """M5 confirmation: candle in trend direction with momentum"""
    df = get_rates(sym, "M5", 50)
    if df is None:
        return False
    a = df.iloc[-2]
    body = abs(a["close"] - a["open"])
    rng = a["high"] - a["low"]
    if rng <= 0:
        return False
    if body / rng < 0.5: # require strong body (>50% of range)
        return False
    if direction == "UP":
        return a["close"] > a["open"]
    return a["close"] < a["open"]

# ----- Position sizing -----
def calc_lot(sym: str, entry: float, sl: float, risk_pct: float) -> float:
    """Compute lot size so that loss at SL == risk_pct of balance."""
    acc = mt5.account_info()
    info = mt5.symbol_info(sym)
    if acc is None or info is None:
        return 0.0

    risk_money = acc.balance * (risk_pct / 100.0)
    sl_dist = abs(entry - sl)
    if sl_dist <= 0:
        return 0.0

    tick_size = info.trade_tick_size or info.point
    tick_value = info.trade_tick_value
    if not tick_size or not tick_value:

```

```

        return 0.0

    money_per_lot = (sl_dist / tick_size) * tick_value
    if money_per_lot <= 0:
        return 0.0

    lot = risk_money / money_per_lot
    step = info.volume_step or 0.01
    lot = round(lot / step) * step
    lot = max(info.volume_min, min(info.volume_max, lot))
    return round(lot, 2)

# ----- Order placement -----
def place_order(sym: str, side: str, lot: float, sl: float, tp: float,
                comment: str) -> Optional[int]:
    info = mt5.symbol_info(sym)
    tick = mt5.symbol_info_tick(sym)
    if info is None or tick is None or lot <= 0:
        return None

    digits = info.digits
    if side == "BUY":
        price, otype = tick.ask, mt5.ORDER_TYPE_BUY
    else:
        price, otype = tick.bid, mt5.ORDER_TYPE_SELL

    req = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": sym, "volume": lot, "type": otype,
        "price": round(price, digits),
        "sl": round(sl, digits),
        "tp": round(tp, digits),
        "deviation": 30,
        "magic": MAGIC_NUMBER,
        "comment": comment[:31],
        "type_time": mt5.ORDER_TIME_GTC,
        "type_filling": mt5.ORDER_FILLING_IOC,
    }
    r = mt5.order_send(req)
    if r is None or r.retcode != mt5.TRADE_RETCODE_DONE:
        msg = getattr(r, "comment", "") if r else mt5.last_error()
        print(f"[!] {sym} {side} failed: {msg}")
        return None
    print(f"[+] {sym} {side} {lot} @ {price:.5f} SL={sl:.5f} TP={tp:.5f} ({comment})")
    return r.order

def modify_sl(pos, new_sl: float) -> bool:
    info = mt5.symbol_info(pos.symbol)
    digits = info.digits if info else 5
    req = {
        "action": mt5.TRADE_ACTION_SLTP,
        "position": pos.ticket,
        "symbol": pos.symbol,
        "sl": round(new_sl, digits),
        "tp": round(pos.tp, digits),
    }
    r = mt5.order_send(req)
    return r is not None and r.retcode == mt5.TRADE_RETCODE_DONE

# ----- Trade state (initial R distance for trailing math) -----
def load_state() -> dict:
    if os.path.exists(STATE_FILE):
        try:
            with open(STATE_FILE) as f:
                return json.load(f)
        except Exception:
            pass
    return {}

def save_state(s: dict):
    try:
        with open(STATE_FILE, "w") as f:

```

```

        json.dump(s, f)
    except Exception as e:
        print(f"[!] state save failed: {e}")

# ----- Open trades count -----
def open_count(sym: Optional[str] = None) -> int:
    pos = mt5.positions_get(symbol=sym) if sym else mt5.positions_get()
    if pos is None:
        return 0
    return sum(1 for p in pos if p.magic == MAGIC_NUMBER)

# ----- Strategy entry pipeline -----
def try_entry(sym: str, state: dict) -> bool:
    if open_count() >= MAX_OPEN_TRADES:
        return False
    if open_count(sym) >= MAX_TRADES_PER_PAIR:
        return False

    sp_cap = MAX_SPREAD_POINTS_CRYPTO if is_crypto(sym) else MAX_SPREAD_POINTS_FX
    if spread_points(sym) > sp_cap:
        return False

    # Tier 1: H4 trend
    direction = htf_trend(sym)
    if direction is None:
        return False
    # Tier 2: H1 strength
    if not h1_strength(sym, direction):
        return False
    # Tier 3: M15 entry zone
    res = m15_entry(sym, direction)
    if res is None:
        return False
    entry, sl, atr_val = res
    # Tier 4: M5 momentum trigger
    if not m5_trigger(sym, direction):
        return False

    side = "BUY" if direction == "UP" else "SELL"
    rr_dist = abs(entry - sl)
    tp = entry + INITIAL_RR * rr_dist if side == "BUY" else entry - INITIAL_RR * rr_dist

    lot = calc_lot(sym, entry, sl, RISK_PERCENT)
    if lot <= 0:
        print(f"[!] {sym} lot calc returned 0 - check tick_value")
        return False

    ticket = place_order(sym, side, lot, sl, tp, f"MTF_{direction}")
    if ticket is None:
        return False

    # Save initial R distance + ATR for trailing
    state[str(ticket)] = {
        "symbol": sym,
        "side": side,
        "entry": entry,
        "initial_sl": sl,
        "r_distance": rr_dist,
        "atr": atr_val,
        "stage": 0,
    }
    # 0 = initial, 1 = BE, 2 = locked, 3 = trailing
    save_state(state)
    return True

def scan_all(state: dict, symbols: list) -> int:
    placed = 0
    for sym in symbols:
        try:
            if try_entry(sym, state):
                placed += 1
        except Exception as e:
            print(f"[!] {sym} scan error: {e}")

```

```
return placed
```

```
# ----- Trade management (smart trailing) -----
def manage_trades(state: dict):
    positions = mt5.positions_get()
    if not positions:
        return

    for pos in positions:
        if pos.magic != MAGIC_NUMBER:
            continue
        st = state.get(str(pos.ticket))
        if st is None:
            continue

        tick = mt5.symbol_info_tick(pos.symbol)
        if tick is None:
            continue

        cur = tick.bid if pos.type == mt5.ORDER_TYPE_BUY else tick.ask
        entry = st["entry"]
        rd = st["r_distance"]      # 1R distance in price
        if rd <= 0:
            continue

        # Current profit in R-multiples
        if pos.type == mt5.ORDER_TYPE_BUY:
            profit_r = (cur - entry) / rd
        else:
            profit_r = (entry - cur) / rd

        if profit_r <= 0:
            continue      # nothing to trail in loss

        new_sl = pos.sl
        stage = st.get("stage", 0)

        # Stage 1: BE at +1R
        if profit_r >= BREAKEVEN_AT_R and stage < 1:
            new_sl = entry
            st["stage"] = 1
            print(f"[~] {pos.symbol} #{pos.ticket} -> BREAKEVEN @ {profit_r:.2f}R")

        # Stage 2: lock half at +1.5R
        if profit_r >= LOCK_HALF_AT_R and stage < 2:
            new_sl = entry + 0.5 * rd if pos.type == mt5.ORDER_TYPE_BUY else entry - 0.5 * rd
            st["stage"] = 2
            print(f"[~] {pos.symbol} #{pos.ticket} -> LOCK +0.5R @ {profit_r:.2f}R")

        # Stage 3: ATR trailing after +2R (Chandelier-style)
        if profit_r >= TRAIL_START_R:
            df = get_rates(pos.symbol, "M15", 50)
            if df is not None:
                atr_now = atr(df, ATR_PERIOD).iloc[-1]
                if pos.type == mt5.ORDER_TYPE_BUY:
                    chandelier = cur - ATR_TRAIL_MULT * atr_now
                    if chandelier > new_sl:
                        new_sl = chandelier
                else:
                    chandelier = cur + ATR_TRAIL_MULT * atr_now
                    if chandelier < new_sl or new_sl == 0:
                        new_sl = chandelier
            st["stage"] = max(stage, 3)

        # Apply only if it improves the SL (never worsen)
        improve = (
            (pos.type == mt5.ORDER_TYPE_BUY and new_sl > pos.sl)
            or (pos.type == mt5.ORDER_TYPE_SELL and (pos.sl == 0 or new_sl < pos.sl))
        )
        if improve and modify_sl(pos, new_sl):
            print(f"    SL moved: {pos.sl:.5f} -> {new_sl:.5f} (R={profit_r:.2f})")

    # Cleanup state for closed positions
    open_tickets = {str(p.ticket) for p in positions if p.magic == MAGIC_NUMBER}
```



```

    for tk in list(state.keys()):
        if tk not in open_tickets:
            state.pop(tk, None)
    save_state(state)

# ----- Main loop -----
def main():
    if not init_mt5():
        return

    # Resolve symbols (handles broker suffixes)
    requested = SYMBOLS_FOREX + SYMBOLS_CRYPTO
    symbols = []
    for s in requested:
        actual = find_symbol(s)
        if actual:
            symbols.append(actual)
            print(f"  [+] {s} -> {actual}")
        else:
            print(f"  [-] {s} not available on this broker")

    if not symbols:
        print("[!] No tradable symbols found.")
        mt5.shutdown()
        return

    state = load_state()
    end = (datetime.now() + timedelta(minutes=RUN_MINUTES)) if RUN_MINUTES > 0 else None
    print(f"\n[*] Running. Risk={RISK_PERCENT}% per trade | Symbols={len(symbols)} | "
          f"'Forever' if end is None else f'Till {end:%H:%M:%S}'")

    try:
        while True:
            if end and datetime.now() >= end:
                print("[*] Time up.")
                break
            placed = scan_all(state, symbols)
            manage_trades(state)
            if placed:
                print(f"[i] Placed {placed} | Open: {open_count()}")
                time.sleep(SCAN_INTERVAL_SEC)
    except KeyboardInterrupt:
        print("\n[*] Stopped by user.")
    finally:
        mt5.shutdown()
        print("[*] MT5 disconnected. Open trades remain managed by SL/TP on broker.")

if __name__ == "__main__":
    main()

```